

★ Exercice 1: Fichiers et descripteurs

Q 1: On considère le programme ci-contre. Compléter le tableau ci-dessous avec les valeurs affichées :

toto.txt existe	oui	oui	non	non
titi.txt existe	oui	non	oui	non
fd2				
errno				

```
1 #include <stdio.h>
2 #include <fcntl.h> /* O_RDONLY */
3 #include <errno.h>
4 int main(){
5     int fd1, fd2;
6     fd1 = open("toto.txt", O_RDONLY, 0);
7     close(fd1);
8     fd2 = open("titi.txt", O_RDONLY, 0);
9     printf("fd2 = %d (errno=%d)\n", fd2, errno);
10    return 0;
11 }
```

Q 2: On suppose que le fichier `toto.txt` contient les six caractères "LAMBDA". On considère les trois programmes ci-après. Qu'affichent chacun de ces programmes?

Ce n'est pas dans le cours, On peut proposer des hypothèses en les justifiant.

Deux open

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 int main() {
4     int fd1, fd2; char c;
5     fd1=open("toto.txt", O_RDONLY, 0);
6     fd2=open("toto.txt", O_RDONLY, 0);
7     read(fd1, &c, 1);
8     read(fd2, &c, 1);
9     printf("c = %c\n", c);
10    return 0;
11 }
```

Après fork

```
1 #include <stdio.h>
2 #include <fcntl.h> /* O_RDONLY */
3 int main(){
4     int fd; char c;
5     fd=open("toto.txt", O_RDONLY, 0);
6     if (fork() == 0) {
7         read(fd, &c, 1);
8         return 0;
9     }
10    wait(NULL);
11    read(fd, &c, 1);
12    printf("c = %c\n", c);
13    return 0;
14 }
```

Après dup

```
1 #include <stdio.h>
2 #include <fcntl.h> /* O_RDONLY */
3 int main(){
4     int fd1, fd2; char c;
5     fd1=open("toto.txt", O_RDONLY, 0);
6     read(fd1, &c, 1);
7     fd2 = dup(fd1);
8     read(fd2, &c, 1);
9     printf("c = %c\n", c);
10    return 0;
11 }
```

★ Exercice 2: Redirections

On veut écrire en C l'équivalent de la commande suivante : `bc < calculs.txt`. Le fichier `calculs.txt` contient des opérations arithmétiques. L'objectif est que le programme redirige l'entrée standard vers ce fichier, puis exécute le programme `bc`.

Q 3: Compléter le code ci-dessous pour obtenir le comportement attendu.

```
int main(void) {

    int fd = ..... //open

    if ( ..... ) {

        perror("open");
        exit(1);
    }

    if (dup2( ..... ) < 0) {

        perror("dup2");
        close(fd);
        exit(2);
    }

    ..... // close
    ..... //execvp

    perror("execvp");
    exit(3);
}
```

★ Exercice 3: Pipes - exo interactif sur ARCHE

En utilisant le schéma étudié en cours, réaliser les commandes suivantes :

Q 4: `ls | wc -l`

Q 5: `ls | xargs wc -l`

Rappel. ici `wc` prend comme arguments la sortie standard de `ls`

★ Exercice 4: Processus et tubes

Soit le programme suivant. Il crée une série de processus ouvrant des tubes entre eux.

Q 6: Dessiner l'organisation des processus et tubes telle qu'elle est en ligne 26. Indiquer quel processus est créé à quel ligne sur le schéma.

Q 7: Décrire en une phrase ce que font les processus créés par ce programme.

```

7 int main(int argc, char *argv[ ]) {
8     int tubA[2], tubB[2], tubC[2];
9     pipe(tubA); pipe(tubB); pipe(tubC);
10    int n1=0, n2=0;
11    close(0); close(1);
12
13    if ((n1=fork())) {
14        dup2(tubA[0],0);    dup2(tubB[1],1);
15    } else if ((n2=fork())) {
16        dup2(tubB[0],0);    dup2(tubC[1],1);
17    } else {
18        dup2(tubC[0],0);    dup2(tubA[1],1);
19        printf("A");
20        fflush(stdout);
21    }
22    close(tubA[0]); close(tubA[1]);
23    close(tubB[0]); close(tubB[1]);
24    close(tubC[0]); close(tubC[1]);
25
26    /* Dessinez l'état des lieux à ce point */
27
28    char c;
29    while ((read(0,&c,1))>0) { /* Cette boucle ne s'arrête jamais */
30        write(1,&c,1);
31    }
32 }
```